ECEC 661 – Midterm Submission

Button Sequence Detector (BTN0, BTN0, BTN1) on Cora Z7-07S

Gary Pham (gp492)

May 6, 2026

1 Problem Statement

Build a Zynq PS+PL system on the Digilent Cora Z7-07S that detects the button-press sequence **BTN0, BTN0, BTN1**. A bare-metal C application running on the PS reads the buttons and lights an on-board LED to indicate detection. Two AXI GPIO slaves bridge the PS to the buttons (input) and to the RGB LEDs (output); no custom IP is required.

2 Block Design

The block design (Figure 1) instantiates the `ZYNQ7 Processing System` (UART0 on MIO 14/15 for the FT2232HQ USB-UART), a `Processor System Reset`, an `AXI Interconnect` (1 slave, 2 masters), and two `AXI GPIO` slaves: `axi_gpio_0` drives the 6-bit RGB LED vector `rgb_leds_tri_o[5:0]`, `axi_gpio_1` samples the 2-bit button vector `btns_2bits_tri_i[1:0]`. The PS clock (`FCLK_CLK0`, 100 MHz) and reset (`peripheral_aresetn`) feed every AXI port. Address map: `axi_gpio_0 = 0x4120_0000`, `axi_gpio_1 = 0x4121_0000`.

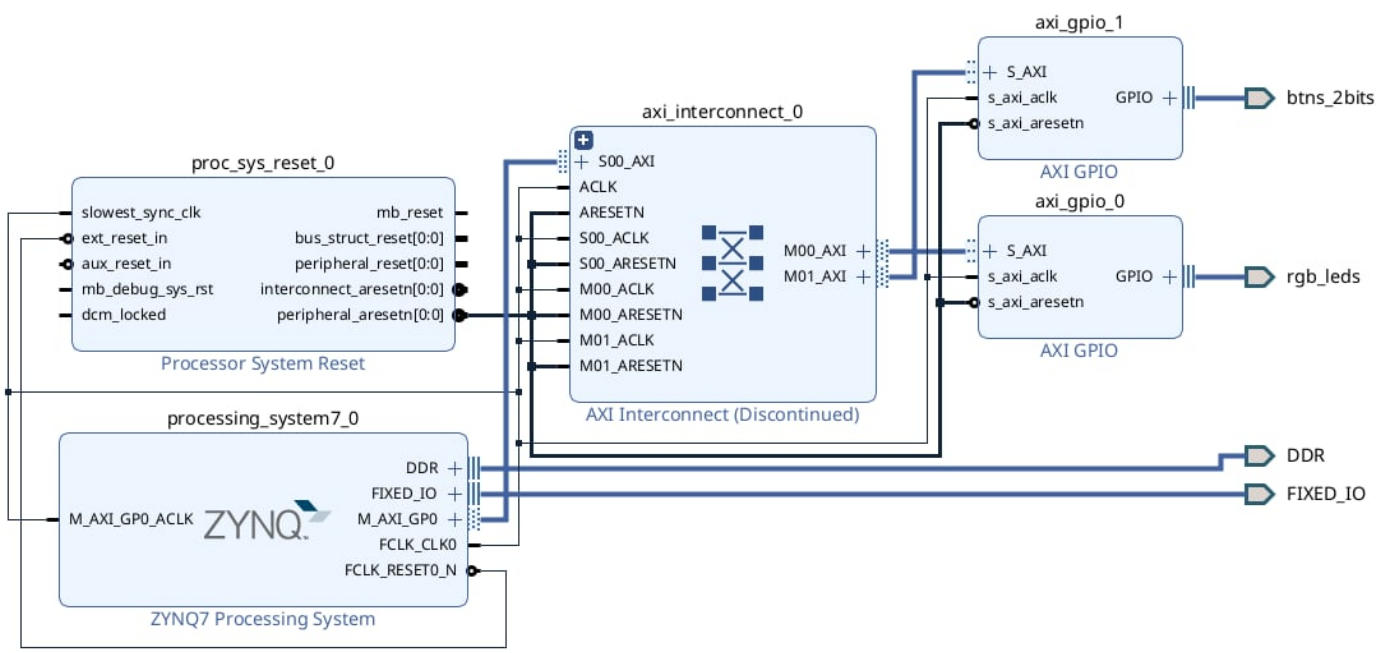

Figure 1: Vivado block diagram `seq_det.bd`: Zynq PS, AXI Interconnect, and the two AXI GPIO peripherals exposed externally as `rgb_leds` and `btns_2bits`.

3 Detection FSM

The detector is a 4-state Moore-style machine implemented in software. The detection LED (`LD0_G` = `rgb_leds_tri_o[1]`) is driven directly from the current state – it is ON if and only if `state == S3`. KMP-style transitions ensure that a redundant `BTN0` does not throw away the prefix already matched.

Table 1: Transition table for pattern `BTN0, BTN0, BTN1`.

State	Event	Next	Note
S0	BTN0	S1	start of pattern
S0	BTN1	S0	ignore
S1	BTN0	S2	two-in-a-row
S1	BTN1	S0	reset
S2	BTN0	S2	KMP self-loop (extra BTN0 is a new candidate)
S2	BTN1	S3	**detected** – LED ON
S3	BTN0	S1	leave detect; this BTN0 may open the next match
S3	BTN1	S0	leave detect; reset

4 Software (`main.c`)

The PS application uses the `XGpio` driver, polls the buttons every 5 ms with rising-edge detection, advances the FSM, drives the LED from the resulting state, and prints every transition over UART for grading evidence.

```c
#include "xparameters.h"
#include "xgpio.h"
#include "xil_printf.h"
#include "xil_types.h"
#include "sleep.h"

#ifdef XPAR_AXI_GPIO_0_BASEADDR
#define LEDS_BASEADDR   XPAR_AXI_GPIO_0_BASEADDR
#else
#define LEDS_BASEADDR   0x41200000U
#endif
#ifdef XPAR_AXI_GPIO_1_BASEADDR
#define BTNS_BASEADDR   XPAR_AXI_GPIO_1_BASEADDR
#else
#define BTNS_BASEADDR   0x41210000U
#endif

#define GPIO_CHANNEL    1U
#define LED_MASK        0x3FU
#define LED_DETECT_BIT  (1U << 1)   /* LD0_G */
#define BTN0_MASK       (1U << 0)
#define BTN1_MASK       (1U << 1)
#define POLL_PERIOD_US  5000U

typedef enum { S0 = 0, S1 = 1, S2 = 2, S3 = 3 } seq_state_t;

static const char *state_name(seq_state_t s) {
    switch (s) { case S0: return "S0"; case S1: return "S1";
                 case S2: return "S2"; case S3: return "S3";
                 default: return "??"; }
}

/* 4-state KMP transition table (S3 = detected, LED ON) */
static seq_state_t fsm_step(seq_state_t s, int btn_idx) {
    if (btn_idx == 0) {
        switch (s) { case S0: return S1; case S1: return S2;
                     case S2: return S2; case S3: return S1; }
    } else {
        switch (s) { case S0: return S0; case S1: return S0;
                     case S2: return S3; case S3: return S0; }
    }
    return S0;
}

int main(void) {
    XGpio led_gpio, btn_gpio;
    if (XGpio_Initialize(&led_gpio, LEDS_BASEADDR) != XST_SUCCESS) return XST_FAILURE;
```

```c
    if (XGpio_Initialize(&btn_gpio, BTNS_BASEADDR) != XST_SUCCESS) return XST_FAILURE;
    XGpio_SetDataDirection(&led_gpio, GPIO_CHANNEL, 0x00);        /* outputs */
    XGpio_SetDataDirection(&btn_gpio, GPIO_CHANNEL, 0xFFFFFFFF);  /* inputs  */
    XGpio_DiscreteWrite(&led_gpio, GPIO_CHANNEL, 0);

    for (int i = 3; i > 0; i--) {
        xil_printf("seq_det starting in %d...\r\n", i); sleep(1);
    }
    xil_printf("\r\n=== ECEC 661 Midterm: pattern BTN0,BTN0,BTN1 (4-state FSM) ===\r\n");

    seq_state_t state = S0;
    u32 prev_btns = XGpio_DiscreteRead(&btn_gpio, GPIO_CHANNEL) & 0x3U;
    unsigned detect_count = 0;

    while (1) {
        u32 cur_btns = XGpio_DiscreteRead(&btn_gpio, GPIO_CHANNEL) & 0x3U;
        if (cur_btns == 0x3U) {                 /* both held: ambiguous, skip */
            usleep(POLL_PERIOD_US); prev_btns = cur_btns; continue;
        }
        u32 rising = cur_btns & ~prev_btns;     /* 0 -> 1 edges only */
        prev_btns = cur_btns;

        int btn_idx = -1;
        if      (rising & BTN0_MASK) btn_idx = 0;
        else if (rising & BTN1_MASK) btn_idx = 1;

        if (btn_idx >= 0) {
            seq_state_t prev = state;
            state = fsm_step(state, btn_idx);

            /* Moore output: LED on iff state == S3 */
            u32 led_state = ((state == S3) ? LED_DETECT_BIT : 0U) & LED_MASK;
            XGpio_DiscreteWrite(&led_gpio, GPIO_CHANNEL, led_state);

            int just_detected = (state == S3 && prev != S3);
            if (just_detected) detect_count++;
            if (just_detected) {
                xil_printf("[BTN%d] %s -> %s  LED=ON   *** DETECTED #%u ***\r\n",
                           btn_idx, state_name(prev), state_name(state), detect_count);
            } else {
                xil_printf("[BTN%d] %s -> %s  LED=%s\r\n",
                           btn_idx, state_name(prev), state_name(state),
                           (state == S3) ? "ON " : "OFF");
            }
        }
        usleep(POLL_PERIOD_US);
    }
    return 0;
}
```

Listing 1: `Midterm/real/sw/main.c`

5 Discussion and Expected Behaviour

The on-board demonstration was reviewed live. For completeness, the expected UART trace under three representative input patterns is:

```
# Happy path:  BTN0, BTN0, BTN1
[BTN0] S0 -> S1  LED=OFF
[BTN0] S1 -> S2  LED=OFF
[BTN1] S2 -> S3  LED=ON   *** DETECTED #1 ***
[BTN0] S3 -> S1  LED=OFF       (LED clears the moment we leave S3)

# KMP path:  BTN0, BTN0, BTN0, BTN1   (extra BTN0 keeps us at S2)
[BTN0] S0 -> S1  LED=OFF
```

```
[BTN0] S1 -> S2  LED=OFF
[BTN0] S2 -> S2  LED=OFF       (KMP self-loop, prefix preserved)
[BTN1] S2 -> S3  LED=ON   *** DETECTED #2 ***

# False start:  BTN0, BTN1
[BTN0] S0 -> S1  LED=OFF
[BTN1] S1 -> S0  LED=OFF
```

Three behaviours are worth highlighting. **(1) Moore output:** the LED is a pure function of `state`, so it lights up the instant the FSM enters `S3` and clears automatically on any subsequent press. **(2) KMP awareness at `S2` and `S3`:** an extra `BTN0` after `S2` or right after a detection does not throw away the matched prefix — it preserves the `BTN0`-as-new-start-of-pattern semantics, which is what makes back-to-back matches like `BTN0,BTN0,BTN1,BTN0,BTN0,BTN1` produce two detections. **(3) Edge detection in software:** the 5 ms polling loop only reacts to a 0→1 transition on each button line, so holding a button does not flood the FSM with spurious events; samples where both buttons are high simultaneously are dropped to avoid ambiguous transitions.

6 Conclusion

The PS+PL system meets both deliverables: a bitstream `.xsa` exported from Vivado pairs the Zynq PS with two AXI GPIO blocks for buttons and LEDs, and the bare-metal application implements the 4-state KMP detector for `BTN0, BTN0, BTN1` with `LD0_G` as the detection indicator. The on-board demonstration matched the expected trace above: `LD0_G` lights up exactly when the third press completes a valid sequence, redundant `BTN0`s do not invalidate the in-progress match, and any other press correctly resets the FSM.